

Medium Data Structure Functions (Solved)

Scope: Non-basic functions only, including ArrayList (Unsorted + Sorted).

1) ArrayList (Unsorted)

Function	Solved idea (medium level)
removeAll(value)	Remove all occurrences of value by compacting non-matching elements.
deduplicate()	Use hash set to keep first appearance and remove duplicates.
rotateLeft(k)	Normalize k, then reverse subranges to rotate in $O(n)$.
moveZerosToEnd()	Two-pointer stable compaction for non-zeros then fill zeros.
maxSubarraySum()	Kadane algorithm in $O(n)$.

Test cases (3 ■■■■■):

Case	Input	Expected
1	[3,1,3,2], removeAll(3)	[1,2]
2	[1,0,2,0,3], moveZerosToEnd()	[1,2,3,0,0]
3	[-2,1,-3,4,-1,2,1,-5,4], maxSubarraySum()	6

2) ArrayList (Sorted)

Function	Solved idea (medium level)
lowerBound(x)	Binary search for first index i where $a[i] \geq x$.
upperBound(x)	Binary search for first index i where $a[i] > x$.
insertSorted(x)	Find lowerBound and shift right once.
deleteAll(x)	Delete range [lowerBound, upperBound).
countInRange(l,r)	$\text{upperBound}(r) - \text{lowerBound}(l)$.

Test cases (3 ■■■■■):

Case	Input	Expected
1	[1,2,2,4], lowerBound(2)	1
2	[1,2,2,4], upperBound(2)	3
3	[1,2,2,4], countInRange(2,4)	3

3) LinkedList (Unsorted)

Function	Solved idea (medium level)
findMiddle()	Use slow/fast pointers; slow ends at middle.
removeNthFromEnd(n)	Advance fast by n, move both, remove target.
isPalindrome()	Reverse second half then compare two halves.
detectCycle()	Floyd tortoise-hare cycle detection.
sortList()	Merge sort specialized for linked list.

Test cases (3 ■■■■■■):

Case	Input	Expected
1	1->2->3->4->5, findMiddle()	3
2	1->2->2->1, isPalindrome()	true
3	1->2->3->4, removeNthFromEnd(2)	1->2->4

4) Stack (Array-based)

Function	Solved idea (medium level)
minElement()	Track current minimum using auxiliary stack.
sortStack()	Use helper stack to keep sorted order.
isBalanced(expr)	Push opening braces and validate closing braces.
evaluatePostfix(expr)	Push operands, apply operator on top two.
nextGreaterElements(arr)	Monotonic decreasing stack of indices.

Test cases (3 ■■■■■■):

Case	Input	Expected
1	expr="{[{()]}"	isBalanced = true
2	postfix="23*54*+9-"	17
3	arr=[2,1,2,4,3]	NGE=[4,2,4,-1,-1]

5) Queue (Array/Linked)

Function	Solved idea (medium level)
reverseQueue()	Use stack or recursion to reverse order.
reverseFirstK(k)	Reverse first k with stack then rotate remaining.
rotate(k)	Move front to rear k times.
interleaveHalves()	Alternate elements from first and second half.
firstNegativeInWindow(k)	Deque stores indices of negative values.

Test cases (3 ■■■■■■):

Case	Input	Expected
1	[1,2,3,4], reverseQueue()	[4,3,2,1]
2	[1,2,3,4,5], reverseFirstK(3)	[3,2,1,4,5]
3	arr=[12,-1,-7,8,-15,30,16,28], k=3	[-1,-1,-7,-15,-15,0]